



Estructuras de datos, lectura y almacenamiento de datos tabulares con Pandas

Series en Pandas

Con los arreglos de NumPy podríamos manejar grandes cantidades de información del área de la salud, siempre y cuando todos los datos que almacenemos sean de tipo numérico; sin embargo, no siempre es así, ya que puede haber datos no numéricos, por ejemplo:

- Los nombres de las vacunas
- Las comorbilidades
- Las entidades federativas o países
- Los nombres de las personas
- Los síntomas

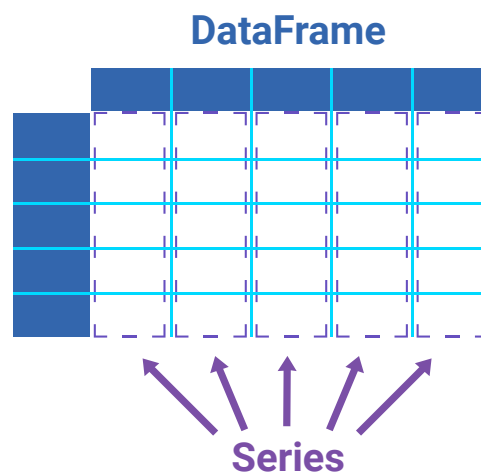
Estos ejemplos corresponden, en todo caso, a caracteres alfanuméricos. Pero, **independientemente del tipo de datos a almacenar, una desventaja de los arreglos es que, para manipularlos y tener acceso a sus elementos, se deben utilizar subíndices enteros.**

Imagina que tienes un arreglo con la cantidad de defunciones por país. ¿Cómo sabrías qué número de índice corresponde a México?, ¿no sería más fácil que pudieras tener acceso con el código del país?, por ejemplo, "MX".

En este sentido, la plataforma Pandas da solución a estos problemas al incluir estructuras especiales con grandes prestaciones en cuanto a diversidad de funciones y facilidad de uso. **Estas estructuras te permiten almacenar información variada respecto a algunos tipos de datos, así como utilizar etiquetas en lugar de subíndices para llegar a estos.**

Estructura de datos

En Pandas, una tabla de datos se denomina **"DataFrame"** y cada columna que la compone se conoce como **"Serie"**.



Las series, en Pandas, son arreglos de una dimensión que pueden estar etiquetados en lugar de emplear un subíndice numérico. En otras palabras, hace referencia a un valor, pues en las series se puede almacenar cualquier tipo de dato: numérico, texto u objetos de Python.



Para definir esta estructura, puedes utilizar el constructor `Series` de Pandas bajo el siguiente procedimiento:

1. Agrega como primer parámetro alguno de los siguientes elementos:
 - Un `ndarray`
 - Un diccionario
 - Un valor escalar
2. Representa los índices de la serie. Este procedimiento es opcional; sin embargo, si se omite, se emplearán por defecto subíndices numéricos del mismo modo que en los arreglos.

Ahora observa cómo crear una serie de Pandas con números aleatorios sin etiquetar.

En el siguiente ejemplo, el constructor `Series` únicamente recibe un parámetro (`rg.integers(10, size=(7,))`) que indica que se generarán 7 números enteros entre 0 y 9. Como no se incluye un segundo parámetro, a cada número aleatorio generado se le asigna por defecto su posición - 1 en la serie como subíndice.

```
import numpy as np
import pandas as pd
rg = np.random.default_rng()
serie = pd.Series(rg.integers(10, size=(7,)))
print("Serie:\n", serie)
```

Serie:

```
0    5
1    4
2    6
3    7
4    3
5    2
6    1
```

dtype: int64

Serie sin etiquetar

Si en el mismo código incluyes el segundo parámetro del constructor `series` para asignar una lista de etiquetas al atributo `index`, entonces tendrás dos formas de acceder a sus elementos: las etiquetas y los índices numéricos, ya que estos últimos seguirán estando disponibles.

```
import numpy as np
import pandas as pd
rg = np.random.default_rng()
serie = pd.Series(rg.integers(10, size=(7,)), index=["1ro",
"2do", "3ro", "4to", "5to", "6to", "7mo"])
print("Serie:\n", serie)
```

Serie:

```
1ro    1
2do    3
3ro    9
4to    8
5to    7
6to    3
7mo    5
```

dtype: int64

Ahora bien, considerando a los diccionarios como herramientas muy útiles y comúnmente empleadas en diversos procesos, es importante resaltar que las **series también pueden ser creadas a partir de un diccionario**, cuando este es pasado como parámetro al constructor, automáticamente la serie creada quedará etiquetada.

```
import pandas as pd
diccionario = {"uno":"I", "dos":"II", "tres":"III",
"cuatro":"IV"}
serie = pd.Series(diccionario)
print("Serie:\n", serie)
```

```
Serie:
uno      I
dos      II
tres     III
cuatro   IV
dtype: object
```

Otra forma en la que puedes crear una serie es a través de un escalar. Para ello, basta con enviarlo al constructor como primer parámetro. De este modo, Pandas automáticamente colocará ese valor en todos los elementos de la serie.

Una vez que tienes datos en la serie, independientemente de la forma en que fue creada, ¿cómo puedes acceder a sus elementos para manipularlos? Para responder a este cuestionamiento, lo primero que deberás preguntarte es si la serie está etiquetada o no.

- **Si está etiquetada, puedes acceder a los elementos a través de sus subíndices o de sus etiquetas.**
- **Si no está etiquetada, únicamente puedes acceder a los elementos a través de sus subíndices.**

Por ejemplo, si trabajas con la siguiente serie sin etiquetar, puedes hacer referencia a sus elementos de la misma forma en que lo haces con los arreglos:

```
import numpy as np
import pandas as pd
rg = np.random.default_rng()
serie = pd.Series(rg.integers(50, size=(7,)))
print("Serie:\n", serie)
```

```
Serie:
0      11
1      14
2      34
3      24
4      30
5      40
6      41
dtype: int64
```

Para acceder a uno de sus elementos, utiliza el subíndice asociado:

```
print("Accesando al segundo elemento a través del
subíndice:", serie[1])
```

Si deseas recuperar valores consecutivos, recuerda que debes indicar el subíndice inicial y el límite, y que este último no se considera. Por ejemplo, recuperando del segundo al cuarto elemento de la serie:

```
print("Obteniendo los elementos del 3er al 6to a través de
un rango:\n", serie[2:5])
```

Obteniendo los elementos del 3er al 6to a través de un rango:

```
2      34
3      24
4      30
```

Para obtener los valores de la serie que cumplen con una condición, los subíndices ya no son considerados. Por ejemplo, si deseas recuperar todos los valores mayores a 15, corresponde lo siguiente:

```
print("Obteniendo los elementos que sean mayores a 15:\n",  
serie[serie>15])
```

Obteniendo los elementos que sean mayores a 15:

2	34
3	24
4	30
5	40
6	41

La última posibilidad es que pases una lista de subíndices específicos para recuperar los valores que se encuentran en esos subíndices, por ejemplo:

```
print("Obteniendo los elementos de los indices 2,4,5 y  
6:\n", serie[[2,4,5,6]])
```

Obteniendo los elementos de los indices 2,4,5 y 6:

2	34
4	30
5	40
6	41

Ahora, imagina que estás trabajando con la siguiente serie etiquetada:

```
import pandas as pd  
serie = pd.Series({"uno":"I", "dos":"II", "tres":"III",  
"cuatro":"IV"})  
print("Serie:\n", serie)
```

Serie:

uno	I
dos	II
tres	III
cuatro	IV

La primera opción, equivalente a las mostradas en la serie no etiquetada, es recuperar un único elemento. Para ello, emplea la etiqueta que le corresponde al valor, por ejemplo:

```
print("Accesando a un elemento de la serie con su  
etiqueta:", serie["tres"])
```

Accesando a un elemento de la serie con su etiqueta: III

Si necesitas recuperar dos valores o más, puedes emplear una lista de etiquetas para recuperar sus elementos correspondientes, por ejemplo:

```
print("Accesando a varios elementos de la serie con una  
lista etiqueta:\n", serie[["dos", "cuatro"]])
```

Accesando a varios elementos de la serie con una lista etiqueta:

```
dos      II  
cuatro   IV
```

Al igual que en los arreglos, con las series también puedes realizar operaciones. Por ejemplo, si quieres aplicarle una función a sus elementos, como el cuadrado, entonces:

```
import numpy as np  
import pandas as pd  
serie = pd.Series([1, 2, 3, 4])  
print("El cuadrado de los elementos de la serie:\n",  
      np.square(serie))
```

El cuadrado de los elementos de la serie:

```
0      1  
1      4  
2      9  
3     16
```


Lo mismo aplica para las operaciones entre series. Por ejemplo, si deseas sumar los elementos de dos series:

```
import pandas as pd
serie = pd.Series([1, 2, 3, 4, 5])
serie2 = pd.Series([10, 20, 30, 40, 50])
print("Suma de series:\n", serie + serie2)
```

Suma de series:

```
0    11
1    22
2    33
3    44
4    55
```

El operador `in` te puede resultar muy útil para hacer búsquedas en las series. Para que lo utilices, indica la llave a buscar, seguida de la palabra `in` y, finalmente, coloca el nombre de la serie donde se está buscando esta llave. Te regresará `True` si lo encuentra o `False` en caso contrario, por ejemplo:

```
import pandas as pd
serie = pd.Series({"uno": "I", "dos": "II", "tres": "III",
                  "cuatro": "IV"})
print("La llave tres se encuentra en la serie: ", "tres" in
serie)
print("La llave ocho se encuentra en la serie: ", "ocho" in
serie)
```

```
La llave tres se encuentra en la serie:  True
La llave ocho se encuentra en la serie:  False
```

Si deseas realizar una búsqueda en los elementos de la serie, debes colocar el elemento a buscar, el operador `in` y, finalmente, el nombre de la serie llamando a su atributo `values`, por ejemplo: `300 in serie.values`

Retomando el ejemplo de almacenar la cantidad de defunciones en varios países, podrías utilizar una estructura donde, en lugar de subíndices, manejes etiquetas que representen al país, es decir, una serie etiquetada, como se muestra a continuación:

```
import pandas as pd
hospitalizadosEstados = pd.Series ([300, 124, 98, 1190],
index=["mx", "cu", "ca", "us"])
hospitalizadosEstados
```

```
mx      300
cu      124
ca       98
us     1190
dtype: int64
```

En el siguiente subtema podrás ver una estructura de datos más completa y avanzada que se utiliza como un estándar en el desarrollo de proyectos de ciencia de datos y que está basada en la Serie.

| **Dataframe en Pandas**

Ahora imagina que quieres combinar información numérica y alfanumérica en la misma estructura, por ejemplo, los datos de las personas que fueron vacunadas para prevenir la influenza estacional y la relación con los nombres de las vacunas que se aplicaron en cuatro países. ¿Qué estructura sería la adecuada?

El *dataframe* es la estructura más importante y la más utilizada en Pandas, posee dos dimensiones de datos etiquetados y sus columnas pueden almacenar diferentes tipos de datos. La mayoría de las veces, esta estructura es llenada con información que proviene de archivos o bases de datos representados de manera tabular. Es importante mencionar que cada columna en un *dataframe* es una serie.

Las maneras más comunes en las que puedes crear un *dataframe* es a partir de:

- Un proceso de lectura de información
- Otro *dataframe*
- Un arreglo de dos dimensiones de NumPy
- Un diccionario de ndarrays de una dimensión, listas, diccionarios o series

Ahora que ya trabajas con series, observa cómo se puede crear un *dataframe* a partir de ellas.

Cada serie corresponderá a una columna en el *dataframe* resultante, y deberás pasarla como parámetro dentro de un diccionario, por ejemplo:

```
import pandas as pd
diccionario = {
    "peso": pd.Series([70, 80, 63], index=["reg1", "reg2",
"reg3"]),
    "estatura": pd.Series([1.77, 1.83, 1.55, 1.63],
index=["reg1", "reg2", "reg3", "reg4"])
}
dataframe = pd.DataFrame(diccionario)
dataframe
```

	peso	estatura
reg1	70.0	1.77
reg2	80.0	1.83
reg3	63.0	1.55
reg4	NaN	1.63

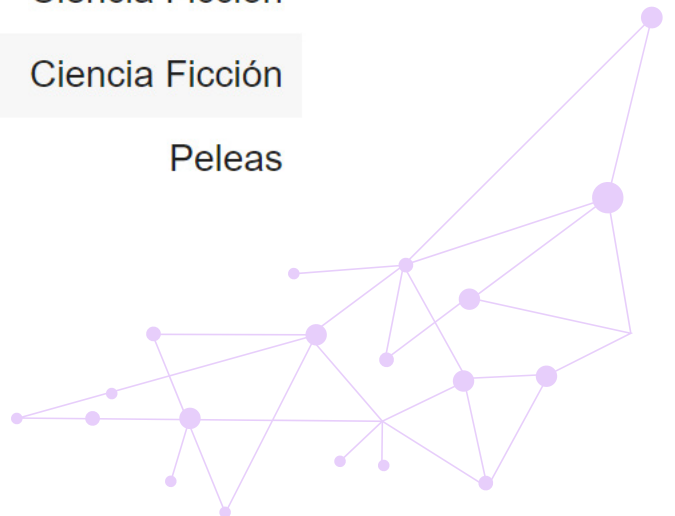
Observa que cada serie se encuentra etiquetada y que la etiqueta corresponde a la fila, conocida como **registro**, donde se colocará cada uno de los datos. Además, si se omite un valor en algún registro existente en el *dataframe* resultante, entonces se colocará NaN en dicho valor.

En el ejemplo anterior, no se pasó el valor del peso para `reg4`, por lo que se le asignó un valor de `NaN`.

Para crear un *dataframe* a partir de un diccionario de listas, se emplean las llaves del diccionario como encabezados de columnas, y los valores en cada lista serán los elementos de las columnas en el *dataframe*. Los valores de cada fila o registro corresponderán a un subíndice entero, por ejemplo, las siguientes listas de películas (descarga el documento desde el apartado “Archivos adjuntos” de la plataforma):

```
import pandas as pd
dataframe = pd.DataFrame(
{"Película": ["Star Wars", "Matrix", "Volver al Futuro",
"Fearless"],
"Año": [1977, 1999, 1985, 2006],
"Genero": ["Ciencia Ficción", "Ciencia Ficción", "Ciencia
Ficción", "Peleas"]}
)
dataframe
```

	Película	Año	Genero
0	Star Wars	1977	Ciencia Ficción
1	Matrix	1999	Ciencia Ficción
2	Volver al Futuro	1985	Ciencia Ficción
3	Fearless	2006	Peleas



Una opción importante es el uso de `set_index` que permite establecer el índice del *dataframe* a partir de alguna columna ya existente. Por ejemplo, si tenemos una copia del *dataframe* anterior en la variable `dataframe2`, podemos observar que su indexación está con base en subíndices enteros de 0 a 3; sin embargo, si queremos que la indexación sea en función de la cantidad de películas de la saga en cuestión, entonces ejecutamos:

```
import pandas as pd
dataframe2 = pd.DataFrame(
{"Pelicula": ["Star Wars", "Matrix", "Volver al Futuro",
"Fearless"],
"Año": [1977, 1999, 1985, 2006],
"Genero": ["Ciencia Ficción", "Ciencia Ficción", "Ciencia
Ficción", "Peleas"], "can_Peliculas": [9, 3, 3, 1]
})
dataframe2
```

	Pelicula	Año	Genero	can_Peliculas
0	Star Wars	1977	Ciencia Ficción	9
1	Matrix	1999	Ciencia Ficción	3
2	Volver al Futuro	1985	Ciencia Ficción	3
3	Fearless	2006	Peleas	1

```
dataframe2.set_index('can_Peliculas')
```

	Pelicula	Año	Genero
can_Peliculas			
9	Star Wars	1977	Ciencia Ficción
3	Matrix	1999	Ciencia Ficción
3	Volver al Futuro	1985	Ciencia Ficción
1	Fearless	2006	Peleas

Para acceder o indexar los elementos de un *dataframe* tienes diferentes posibilidades. Entre las formas más comunes se encuentran las siguientes:

Tipo de acceso	Resultado
Seleccionar una columna	Serie
Seleccionar varias columnas	DataFrame
Seleccionar filas por condición	DataFrame
Seleccionar filas por segmentación	DataFrame
Seleccionar filas y columnas por condición y etiqueta	DataFrame
Seleccionar filas y columnas por su subíndice	DataFrame

Por ejemplo, si necesitas seleccionar los datos contenidos en una de las columnas del *dataframe*, lo único que debes hacer es utilizar el nombre del *dataframe* y agregar entre paréntesis cuadrados el nombre de la columna que tiene los datos requeridos. El siguiente fragmento de código contiene la recuperación de la columna "Película" del *dataframe*:

```
import pandas as pd
dataframe = pd.DataFrame(
{"Pelicula": ["Star Wars", "Matrix", "Volver al Futuro",
"Fearless"],
"Año": [1977, 1999, 1985, 2006],
"Genero": ["Ciencia Ficción", "Ciencia Ficción", "Ciencia
Ficción", "Peleas"]
})
dataframe["Pelicula"]

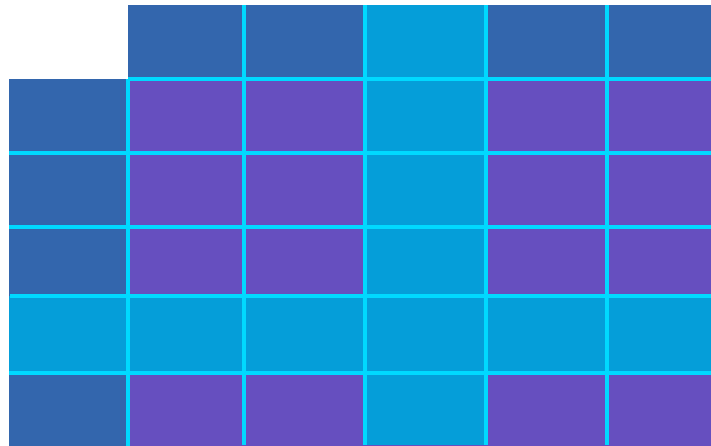
0          Star Wars
1           Matrix
2  Volver al Futuro
3          Fearless
Name: Pelicula, dtype: object
```

Como puedes observar, el resultado anterior es una serie. El resto de los tipos de acceso se verán detalladamente en el próximo tema.

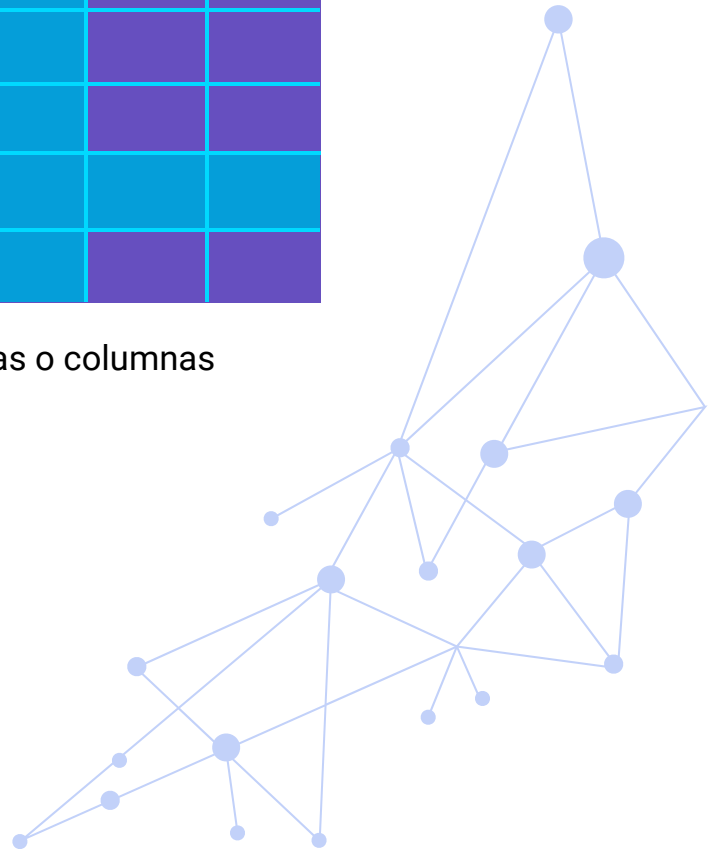
Operaciones con las filas y columnas de un *dataframe*

Cuando trabajas con un *dataframe* es posible que necesites realizar las siguientes operaciones:

- Agregar columnas a un *dataframe*
- Eliminar columnas en un *dataframe*
- Agregar filas o registros a un *dataframe*
- Eliminar filas o registros a un *dataframe*



Trabajando con filas o columnas



A continuación, te mostramos en qué consiste cada una de estas operaciones.

■ Agregar columnas a un *dataframe*

Para agregar columnas a un *dataframe* a partir de los datos de una serie, lo único que debes hacer es asignar la serie al *dataframe* en una columna específica. En otras palabras, utiliza un nombre de columna que no exista, dentro de los paréntesis cuadrados. Por ejemplo, si quieres agregar los datos de la cantidad de películas que tenemos en una serie llamada `can_Peliculas`, se asignan dichos valores al *dataframe*, especificando entre paréntesis cuadrados el nuevo nombre de la columna en el *dataframe*:

```
import pandas as pd
dataframe = pd.DataFrame(
{"Pelicula": ["Star Wars", "Matrix", "Volver al Futuro",
"Fearless"],
"Año": [1977, 1999, 1985, 2006],
"Genero": ["Ciencia Ficción", "Ciencia Ficción", "Ciencia
Ficción", "Peleas"]
})
can_Peliculas = pd.Series([9, 3, 3, 1])
dataframe["can_Peliculas"] = can_Peliculas
dataframe
```

	Pelicula	Año	Genero	can_Peliculas
0	Star Wars	1977	Ciencia Ficción	9
1	Matrix	1999	Ciencia Ficción	3
2	Volver al Futuro	1985	Ciencia Ficción	3
3	Fearless	2006	Peleas	1

■ Eliminar columnas en un *dataframe*

Ahora, si lo que necesitas es eliminar una columna, tienes dos formas:

1. Utilizar la instrucción `del` sobre el *dataframe*, indicando la columna que se desea eliminar. Dicha instrucción afectará directamente el *dataframe* eliminando la columna solicitada. Por ejemplo, para eliminar la columna **"año"**:

```
import pandas as pd
dataframe = pd.DataFrame(
{"Pelicula": ["Star Wars", "Matrix", "Volver al Futuro",
"Fearless"],
"Año": [1977, 1999, 1985, 2006],
"Genero": ["Ciencia Ficción", "Ciencia Ficción", "Ciencia
Ficción", "Peleas"]
})
del dataframe["Año"]
dataframe
```

	Pelicula	Genero
0	Star Wars	Ciencia Ficción
1	Matrix	Ciencia Ficción
2	Volver al Futuro	Ciencia Ficción
3	Fearless	Peleas

2. Utilizar la función `drop`, la cual no afecta el *dataframe* original, sino que genera uno nuevo sin la columna deseada. Para eliminar los datos permanentemente en el *dataframe* original, habría que asignarle el *dataframe* generado o hacer uso del parámetro `inplace=True`.

La función `drop` recibe como parámetros una lista con el nombre de las columnas a eliminar y la propiedad `axis` igualada a 1, indicando que se van a eliminar columnas, por ejemplo:

```
import pandas as pd
dataframe = pd.DataFrame(
{"Pelicula": ["Star Wars", "Matrix", "Volver al Futuro",
"Fearless"],
"Año": [1977, 1999, 1985, 2006],
"Genero": ["Ciencia Ficción", "Ciencia Ficción", "Ciencia
Ficción", "Peleas"]
})
dataframe = dataframe.drop(["Año"], axis=1)
dataframe
```

	Pelicula	Genero
0	Star Wars	Ciencia Ficción
1	Matrix	Ciencia Ficción
2	Volver al Futuro	Ciencia Ficción
3	Fearless	Peleas

■ Agregar filas o registros a un *dataframe*

Una forma de agregar un registro (fila) en un *dataframe* es haciendo uso de la función `concat`, la cual permite unir o concatenar dos *dataframes*, por lo que, si deseamos agregar un registro o varios, deben estar contenidos en un *dataframe*.

En el siguiente ejemplo se crea un *dataframe* llamado “nuevo”, con el registro que se desea agregar al *dataframe* que contiene todas las otras películas. Después se hace uso de la función `concat`, donde el primer parámetro es la lista de *dataframes* a concatenar; el segundo es la opción `ignore_index=True`.

```
import pandas as pd
dataframe = pd.DataFrame(
{"Película": ["Star Wars", "Matrix", "Volver al Futuro",
"Fearless"],
"Año": [1977, 1999, 1985, 2006],
"Genero": ["Ciencia Ficción", "Ciencia Ficción", "Ciencia
Ficción", "Peleas"]
})
nuevo = pd.DataFrame({"Película":["Enemigo al Acecho"],
"Año":[2001], "Genero":["Bélica"]})
dataframe = pd.concat([dataframe, nuevo], ignore_index=True)
dataframe
```

	Película	Año	Genero
0	Star Wars	1977	Ciencia Ficción
1	Matrix	1999	Ciencia Ficción
2	Volver al Futuro	1985	Ciencia Ficción
3	Fearless	2006	Peleas
4	Enemigo al Acecho	2001	Bélica

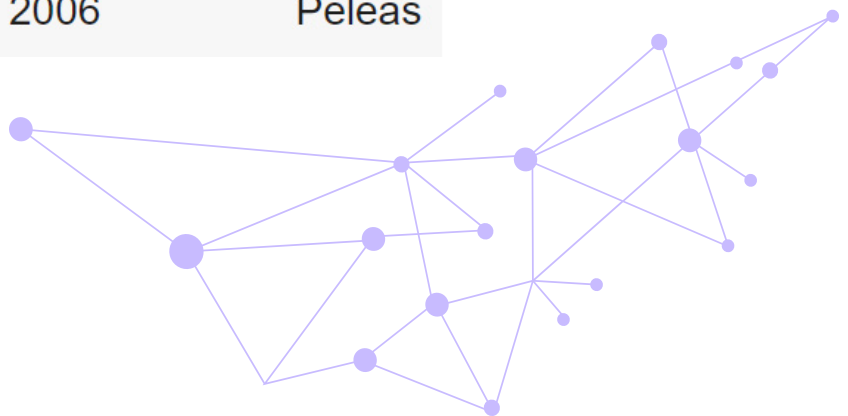
Cuando agregamos registros a un *dataframe* es importante tener en cuenta el parámetro `ignore_index`. Este parámetro, por defecto, está en `False` e indica que, cuando se agregan registros desde otro *dataframe*, se pueden utilizar los índices del nuevo *dataframe*. El problema es que pueden estar ya ocupados en el *dataframe* al que le vamos a agregar las filas. Si lo colocas en `True`, aseguras que los índices de los nuevos datos sean una continuación de los que ya se encuentran en el *dataframe* donde quedará toda la información.

■ Eliminar filas o registros a un *dataframe*

Cuando necesites eliminar todos los datos de un registro en particular, puedes utilizar también la función `drop`. En este caso, debes emplear como parámetros una lista con los índices de los registros a eliminar (o las etiquetas en caso de que los registros se encuentren etiquetados) y la propiedad `axis` igualada a 0, indicando que se van a eliminar renglones. Por ejemplo, para eliminar todo el registro en el subíndice 2 empleamos:

```
import pandas as pd
dataframe = pd.DataFrame(
{"Pelicula": ["Star Wars", "Matrix", "Volver al Futuro",
"Fearless"],
"Año": [1977, 1999, 1985, 2006],
"Genero": ["Ciencia Ficción", "Ciencia Ficción", "Ciencia
Ficción", "Peleas"]
})
dataframe = dataframe.drop([2], axis=0)
dataframe
```

	Pelicula	Año	Genero
0	Star Wars	1977	Ciencia Ficción
1	Matrix	1999	Ciencia Ficción
3	Fearless	2006	Peleas



Retomando el ejemplo inicial, podemos manejar en un *dataframe* la cantidad de vacunados y el nombre de la(s) vacuna(s) de cada país. Por ejemplo, podríamos tener lo siguiente:

```
import pandas as pd
vacunacion = pd.DataFrame(
{"CantidadVacunados" : pd.Series([5000000, 350000000,
15000000, 2000000], index=['México', 'Estados Unidos',
"Canada", "Cuba"]),
"Vacunas" : pd.Series(["Pfizer y cansino", "Pfizer, Sinovac",
"Sinovac", "Cansino"], index=['México', 'Estados Unidos',
"Canada", "Cuba"])}
vacunacion
```

	CantidadVacunados	Vacunas
México	5000000	Pfizer y cansino
Estados Unidos	350000000	Pfizer, Sinovac
Canada	15000000	Sinovac
Cuba	2000000	Cansino

En este subtema pudiste conocer y entender la estructura de datos principal de los proyectos de ciencia de datos (el *dataframe*), desde su creación hasta su manipulación básica de filas y columnas; sin embargo, el proceso de creación normalmente no se realiza, ya que el *dataframe* se construye a partir de un proceso de lectura o generación de información. En el siguiente subtema podrás ver las funciones especializadas de Pandas para realizar la lectura de datos de archivos que dejan la información en un *dataframe*.

Leyendo y escribiendo datos tabulares con Pandas

Seguramente en tus proyectos de ciencia de datos no ingresas la información de manera manual, ni tienes usuarios capturando dato por dato. En este sentido, y regresando a nuestro ejemplo sobre los datos de salud, tendrás la información de vacunación en un archivo de Excel u otro formato. Entonces, ¿cómo puedes obtener la información contenida en un archivo e incorporarla en tus desarrollos?

Diversa información almacenada tiene estructura tabular y orígenes de datos diversos. De ahí la necesidad de conocer procesos de lectura y escritura para los *scripts* de análisis de dicha información.

Conocer cómo trabajar con datos tabulares será fundamental para que puedas manipular el contenido de tus archivos y bases de datos. La similitud en la representación de un *dataframe* de Pandas con una hoja de cálculo o una tabla de SQL y su rapidez y eficiencia, contribuirán al éxito de tus desarrollos.

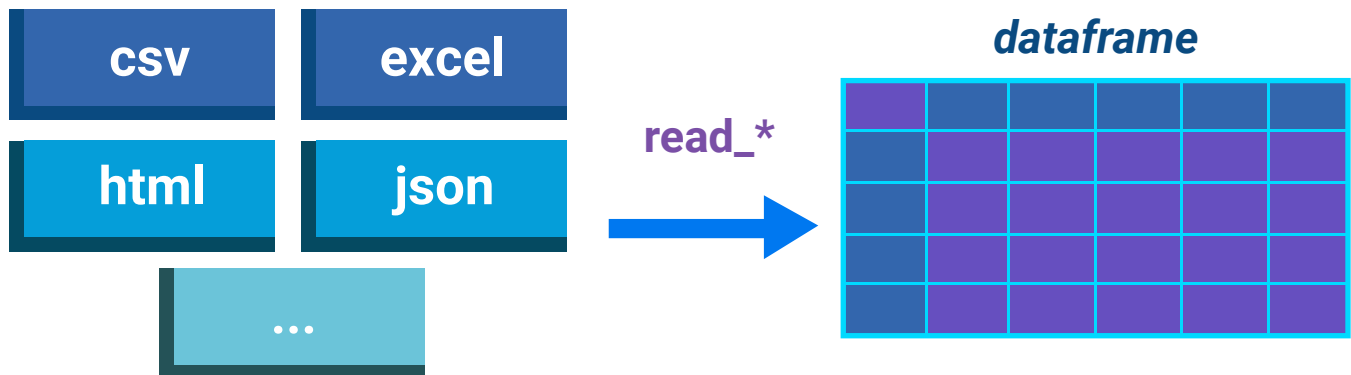
La lectura de datos tabulares es la forma más frecuente de encontrar y obtener información, independientemente de su origen, ya sean archivos, bases de datos o formato. Para esta tarea, la plataforma Pandas tiene varias ventajas, como, por ejemplo:

- Intercambio ágil de datos entre el almacenamiento permanente y memoria
- Indexación integrada
- Manejo de datos faltantes
- Operaciones sobre conjuntos de datos

Ahora bien, puedes realizar la lectura de la información tabular en la plataforma Pandas a través de las funciones conocidas como `read_*`.

Donde el * representa el formato del origen de la información. Por ejemplo:

- Si quieres leer de un archivo separado por comas, utiliza `read_csv`.
- Si es un archivo de Excel, utiliza `read_excel`.



Te recomendamos consultar el material adicional de este documento para que explores la documentación oficial de Pandas para entender todos los orígenes de datos de los cuales se puede extraer la información. Las funciones disponibles son:

```
read_clipboard
read_csv
read_excel
read_feather
read_fwf
read_gbq
read_hdf
read_html
read_json
read_orc
read_parquet
read_pickle
```

Imagina que tienes un archivo llamado **"characters.csv"** (descárgalo desde el apartado "Archivos adjuntos" de la plataforma) con la información de los personajes de *Star Wars* separados por comas. Su lectura es muy sencilla, ya que se hace llamando a la función `read_csv`, y el resultado queda almacenado en un *dataframe*. Por ejemplo:

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
print(personajesSW)
```

	id	name	height	mass	...	birth_year	gender	homeworld	species
0	1	Luke Skywalker	172	77.0	...	19BBY	male	Tatooine	Human
1	2	C-3PO	167	75.0	...	112BBY	NaN	Tatooine	Droid
2	3	R2-D2	96	32.0	...	33BBY	NaN	Naboo	Droid
3	4	Darth Vader	202	136.0	...	41.9BBY	male	Tatooine	Human
4	5	Leia Organa	150	49.0	...	19BBY	female	Alderaan	Human
..	..	...	...	...	...	...	...	...	...
82	83	Rey	168	77.0	...	NaN	female	NaN	Human
83	84	Poe Dameron	175	81.0	...	NaN	male	NaN	Human
84	85	BB8	45	60.0	...	NaN	none	NaN	Droid
85	86	Captain Phasma	170	79.0	...	NaN	female	NaN	NaN
86	87	Padmé Amidala	165	45.0	...	46BBY	female	Naboo	Human

[87 rows x 11 columns]

Como puedes observar, se leyeron 87 registros y se presentaron los primeros y últimos al imprimir el *dataframe*. Además, la ejecución informa que hay 11 columnas, que en este ejemplo no se visualizan por cuestiones de espacio.

Ahora bien, una vez que empiezas a tener *dataframes* con grandes cantidades de datos, es importante que conozcas su estructura para manipularlos adecuadamente.

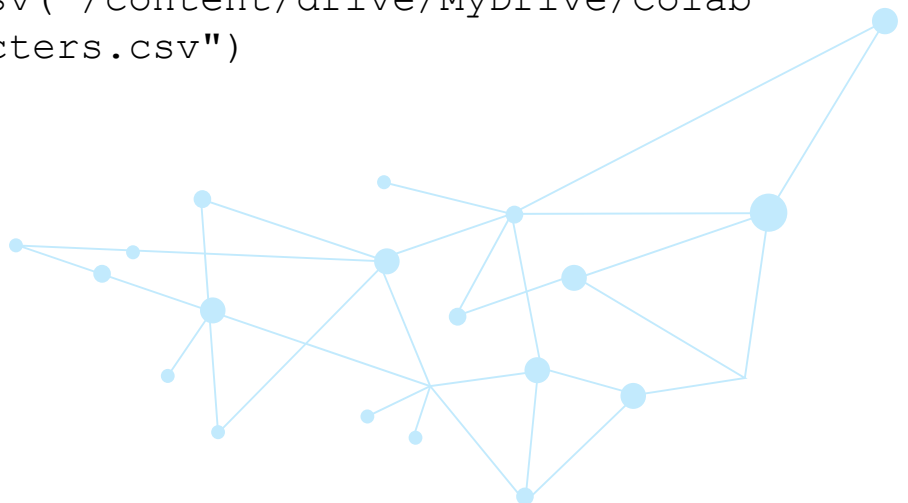
La función `head` te permite mostrar los primeros registros de un *dataframe*, indicando como parámetro la cantidad de registros a visualizar. Si se omite esta cantidad, presenta por defecto los primeros 5 registros. Observa el ejemplo:

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
personajesSW.head(8)
```

	id	name	height	mass	hair_color	skin_color	eye_color	birth_year	gender	homeworld	species
0	1	Luke Skywalker	172	77.0	blond	fair	blue	19BBY	male	Tatooine	Human
1	2	C-3PO	167	75.0	NaN	gold	yellow	112BBY	NaN	Tatooine	Droid
2	3	R2-D2	96	32.0	NaN	white, blue	red	33BBY	NaN	Naboo	Droid
3	4	Darth Vader	202	136.0	none	white	yellow	41.9BBY	male	Tatooine	Human
4	5	Leia Organa	150	49.0	brown	light	brown	19BBY	female	Alderaan	Human
5	6	Owen Lars	178	120.0	brown, grey	light	blue	52BBY	male	Tatooine	Human
6	7	Beru Whitesun Lars	165	75.0	brown	light	blue	47BBY	female	Tatooine	Human
7	8	R5-D4	97	32.0	NaN	white, red	red	NaN	NaN	Tatooine	Droid

En cambio, la función `tail` trabaja de la misma manera que `head`, pero con los últimos registros del *dataframe*:

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
personajesSW.tail(7)
```



	id	name	height	mass	hair_color	skin_color	eye_color	birth_year	gender	homeworld	species
80	81	Tion Medon	206	80.0	none	grey	black	NaN	male	Utapau	Pau'an
81	82	Finn	177	87.0	black	dark	dark	NaN	male	NaN	Human
82	83	Rey	168	77.0	brown	light	hazel	NaN	female	NaN	Human
83	84	Poe Dameron	175	81.0	brown	light	brown	NaN	male	NaN	Human
84	85	BB8	45	60.0	none	none	black	NaN	none	NaN	Droid
85	86	Captain Phasma	170	79.0	NaN	NaN	NaN	NaN	female	NaN	NaN
86	87	Padmé Amidala	165	45.0	brown	light	brown	46BBY	female	Naboo	Human

Si después de recuperar información quieres saber qué tipos de datos se están considerando en las columnas del *dataframe* que resulta del proceso de lectura, debes utilizar el atributo `dtypes`. Si deseas que, además de esta información, se incluya la cantidad de registros y columnas, así como el conteo de no nulos y el tipo de dato, entonces es mejor utilizar la función `info`, como se observa a continuación:

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
```

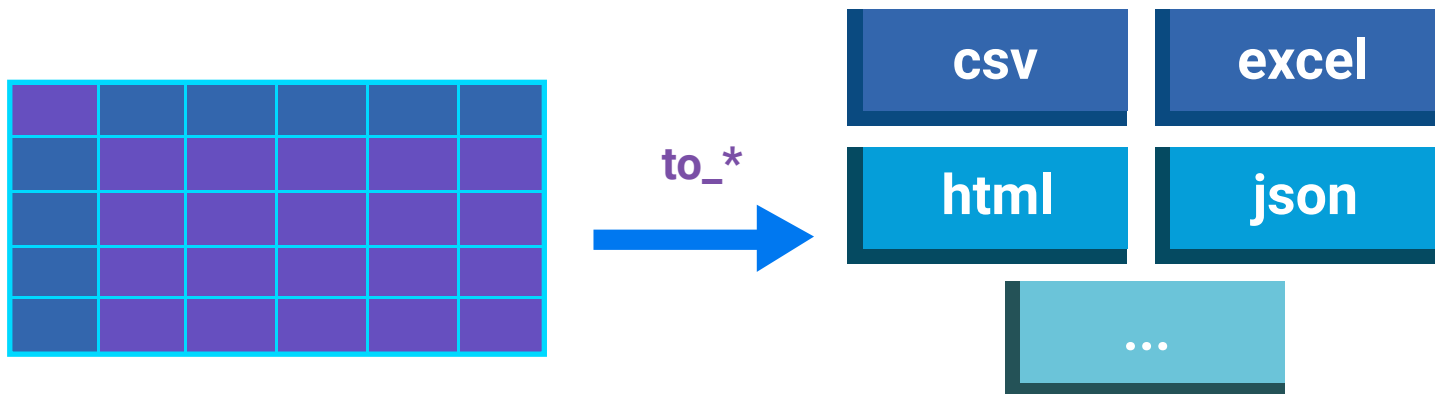
PersonajesSW.dtypes

```
dtypes
id          int64
name        object
height      int64
mass        float64
hair_color  object
skin_color  object
eye_color   object
birth_year  object
gender      object
homeworld   object
species     object
dtype: object
```

personajesSW.info()

```
Info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 87 entries, 0 to 86
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0   id               87 non-null    int64
1   name             87 non-null    object
2   height           87 non-null    int64
3   mass             87 non-null    float64
4   hair_color       81 non-null    object
5   skin_color       85 non-null    object
6   eye_color        84 non-null    object
7   birth_year       43 non-null    object
8   gender           84 non-null    object
9   homeworld        77 non-null    object
10  species          82 non-null    object
dtypes: float64(1), int64(2), object(8)
memory usage: 7.6+ KB
```

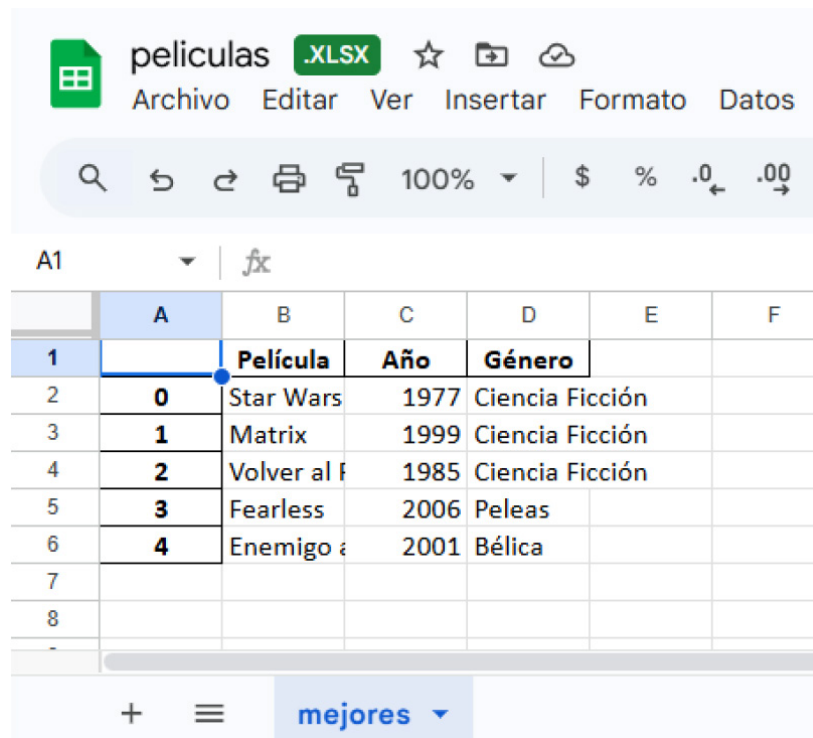
Una vez que has procesado el *dataframe*, modificando su estructura, cambiando la cantidad de datos que contiene y realizando cálculos o resúmenes, es muy probable que desees almacenar estos cambios para hacerlos persistentes y no tener que realizar los procesos una y otra vez. En este sentido, Pandas suministra mecanismos para guardar el *dataframe* en diferentes formatos. Para ello, existen las funciones `to_*`, donde el `*` denota el formato que utilizarás para guardar la información. Toma en cuenta que los formatos de estas son los mismos que las funciones `read_*`



Si deseas guardar en un archivo de Excel la lista de películas generada en el *dataframe* de los ejemplos anteriores, entonces debes utilizar la función `to_excel`. Esta función recibe como parámetro el nombre del archivo con su ruta donde desees almacenar la información; y como segundo parámetro, el atributo `sheet_name` igualado al nombre de la hoja donde se escribirán los datos:

```
import pandas as pd
dataframe = pd.DataFrame(
{"Pelicula": ["Star Wars", "Matrix", "Volver al Futuro",
"Fearless", "Enemigo al Acecho"],
"Año": [1977, 1999, 1985, 2006, 2001],
"Genero": ["Ciencia Ficción", "Ciencia Ficción", "Ciencia
Ficción", "Peleas", "Bélica"]}
)
dataframe.to_excel("/content/drive/MyDrive/Colab Notebooks/
M2DCData/peliculas.xlsx", sheet_name="mejores")
```

El archivo de Excel resultante se verá de la siguiente manera:



The screenshot shows an Excel spreadsheet titled 'películas' with a menu bar (Archivo, Editar, Ver, Insertar, Formato, Datos) and a toolbar. The spreadsheet has columns A through F. Column A contains an index from 0 to 4. Column B contains movie titles: 'Star Wars', 'Matrix', 'Volver al futuro', 'Fearless', and 'Enemigo público'. Column C contains years: '1977', '1999', '1985', '2006', and '2001'. Column D contains genres: 'Ciencia Ficción', 'Ciencia Ficción', 'Ciencia Ficción', 'Peñas', and 'Bélica'. Columns E and F are empty.

	A	B	C	D	E	F
1		Película	Año	Género		
2	0	Star Wars	1977	Ciencia Ficción		
3	1	Matrix	1999	Ciencia Ficción		
4	2	Volver al futuro	1985	Ciencia Ficción		
5	3	Fearless	2006	Peñas		
6	4	Enemigo público	2001	Bélica		
7						
8						
9						

Es importante mencionar que, bajo esta forma de trabajo, si tratas de escribir varios *dataframes* en el mismo archivo, aunque coloques diferentes nombres de hojas (*sheet_name*) en la escritura, los datos serán sobrescritos y solamente se quedarán almacenados los últimos; todos los demás se perderán. Para resolver este problema se utiliza la función `ExcelWriter`, conocida como **“writer”**, la cual permite establecer un flujo de salida que se utiliza para direccionar el almacenamiento a un mismo archivo sin perder los datos ni sobrescribir información. Al llamar a esta función, se coloca como parámetro el nombre del archivo de salida. Después, en las llamadas de almacenamiento de cada *dataframe*, con la función `to_excel` se pasa como parámetro el nombre del **writer** y el nombre de la hoja donde se desea almacenar esa información. Por ejemplo:

```
with pd.ExcelWriter('/content/datos.xlsx') as writer:
    peliculas.to_excel(writer, sheet_name="peliculas")
    personajesSW.to_excel(writer, sheet_name="personajes")
```

Autoguardado

Inicio Insertar Dibujar Disposición

Pegar

Calibri (Cuerpo) 11

N K S

A1

	A	B	C	D	E
1		Pelicula	Año	Genero	
2	0	Star Wars	1977	Ciencia Ficción	
3	1	Matrix	1999	Ciencia Ficción	
4	2	Volver al Futuro	1985	Ciencia Ficción	
5	3	Fearless	2006	Peleas	
6	4	Enemigo al	2001	Bélica	
7					
8					
9					

peliculas personajes +

Autoguardado

Inicio Insertar Dibujar Disposición de página Fórmulas Datos Revisar Vista Automatización

Pegar

Calibri (Cuerpo) 11

N K S

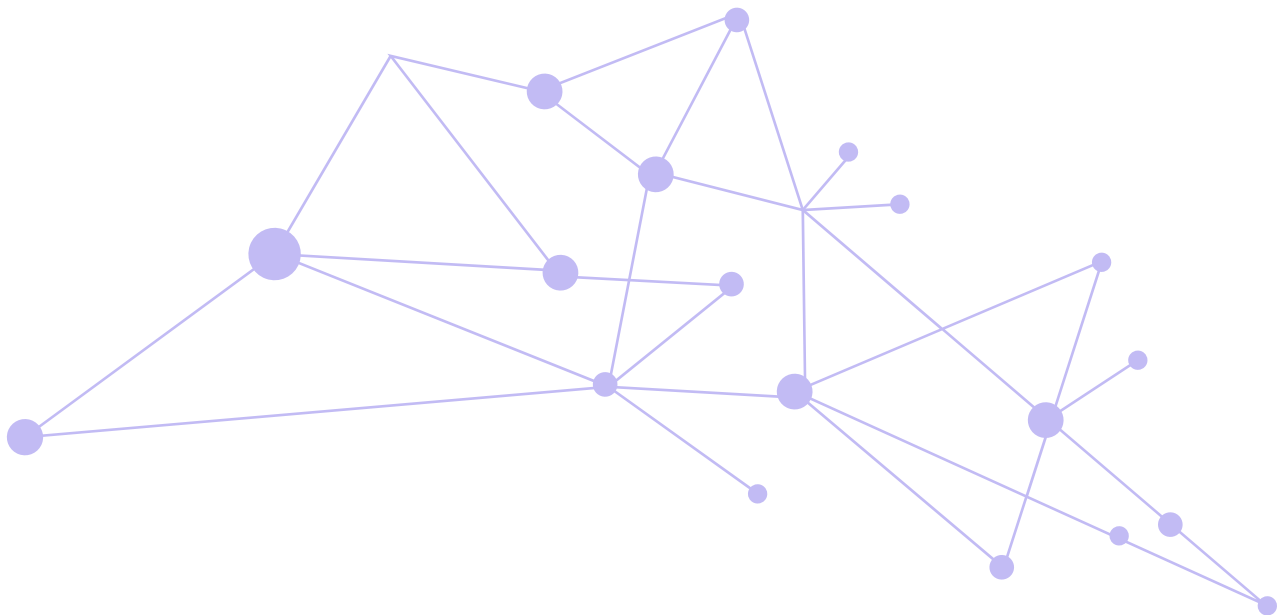
A1

	A	B	C	D	E	F	G	H	I	J	K	L	M
1		id	name	height	mass	hair_color	skin_color	eye_color	birth_year	gender	homeworld	species	
2	0	1	Luke Skywalker	172	77	blond	fair	blue	19BBY	male	Tatooine	Human	
3	1	2	C-3PO	167	75		gold	yellow	112BBY		Tatooine	Droid	
4	2	3	R2-D2	96	32		white, blue	red	33BBY		Naboo	Droid	
5	3	4	Darth Vader	202	136	none	white	yellow	41.9BBY	male	Tatooine	Human	
6	4	5	Leia Organa	150	49	brown	light	brown	19BBY	female	Alderaan	Human	
7	5	6	Owen Lars	178	120	brown, grey	light	blue	52BBY	male	Tatooine	Human	
8	6	7	Beru White	165	75	brown	light	blue	47BBY	female	Tatooine	Human	
9	7	8	R5-D4	97	32		white, red	red			Tatooine	Droid	

peliculas **personajes** +

Ahora veamos un ejemplo para colocar los datos del *dataframe* en una página *web*. Cuando se emplea la función `to_html`, la información del *dataframe* se incluye en una tabla en formato HTML. Después, podrías simplemente aplicarle estilos y complementarla con texto para generar una página informativa. Por ejemplo:

```
import pandas as pd
dataframe = pd.DataFrame(
{"Pelicula": ["Star Wars", "Matrix", "Volver al Futuro",
"Fearless", "Enemigo al Acecho"],
"Año": [1977, 1999, 1985, 2006, 2001],
"Genero": ["Ciencia Ficción", "Ciencia Ficción", "Ciencia
Ficción", "Peleas", "Bélica"]}
)
dataframe.to_html("/content/drive/MyDrive/Colab Notebooks/
M2DCData/peliculas.html")
```



El código anterior produciría el siguiente resultado:

```
peliculas.html x
<table border="1" class="dataframe">
  <thead>
    <tr style="text-align: right;">
      <th></th>
      <th>Película</th>
      <th>Año</th>
      <th>Genero</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th>0</th>
      <td>Star Wars</td>
      <td>1977</td>
      <td>Ciencia Ficción</td>
    </tr>
    <tr>
      <th>1</th>
      <td>Matrix</td>
      <td>1999</td>
      <td>Ciencia Ficción</td>
    </tr>
    <tr>
      <th>2</th>
      <td>Volver al Futuro</td>
      <td>1985</td>
      <td>Ciencia Ficción</td>
    </tr>
    <tr>
      <th>3</th>
      <td>Fearless</td>
      <td>2006</td>
      <td>Peleas</td>
    </tr>
    <tr>
      <th>4</th>
      <td>Enemigo al Acecho</td>
      <td>2001</td>
      <td>Bélica</td>
    </tr>
  </tbody>
</table>
```

Código HTML

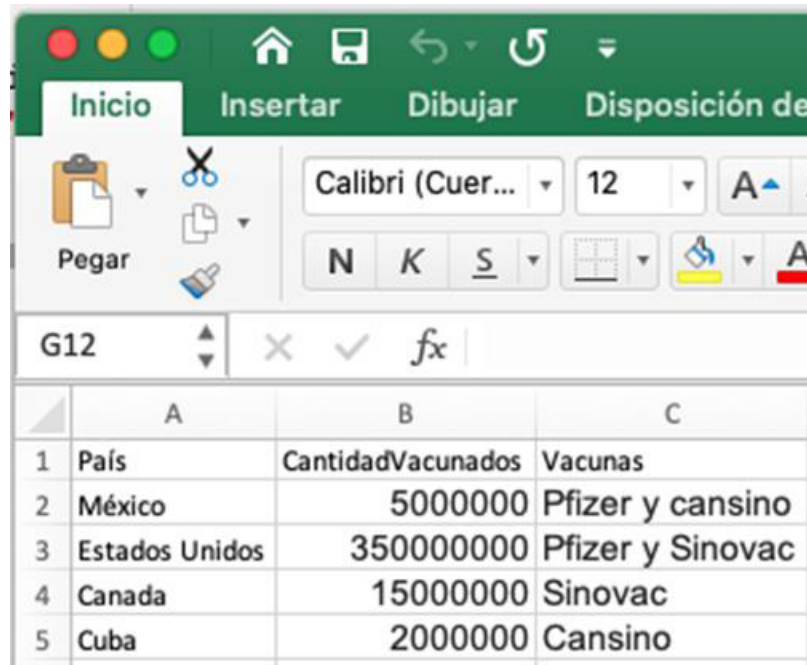
	Película	Año	Genero
0	Star Wars	1977	Ciencia Ficción
1	Matrix	1999	Ciencia Ficción
2	Volver al Futuro	1985	Ciencia Ficción
3	Fearless	2006	Peleas
4	Enemigo al Acecho	2001	Bélica

Visualizándolo en un navegador web

Retomando nuevamente nuestro ejemplo inicial, podrías cargar los datos de la vacunación desde un archivo de Excel, asumiendo que se llama “**vacunación.xlsx**”, de la siguiente manera:

```
import pandas as pd
vacunacion = pd.read_excel("vacunacion.xlsx")
vacunacion
```

	País	CantidadVacunados	Vacunas
0	México	5000000	Pfizer y cansino
1	Estados Unidos	350000000	Pfizer y Sinovac
2	Canada	15000000	Sinovac
3	Cuba	2000000	Cansino



The screenshot shows the Microsoft Excel interface with the 'Inicio' (Home) ribbon selected. The ribbon includes options for 'Pegar' (Paste), 'Calibri (Cuer...)' (Font), '12' (Size), 'A' (Style), 'N' (Number), 'K' (Text), 'S' (Text), and 'fx' (Formulas). The active cell is G12. Below the ribbon, a table is visible with the following data:

	A	B	C
1	País	CantidadVacunados	Vacunas
2	México	5000000	Pfizer y cansino
3	Estados Unidos	350000000	Pfizer y Sinovac
4	Canada	15000000	Sinovac
5	Cuba	2000000	Cansino

Pandas proporciona una amplia variedad de funciones para la lectura y escritura de datos tabulares en diversos formatos, lo que nos permite centrarnos en la manipulación y procesamiento de la información.

| Antes de terminar...

Las series y los *dataframes* de Pandas te permiten manipular información de manera sencilla e intuitiva, mediante diversos métodos. Además, son excelentes estructuras de datos de alto rendimiento para manipular grandes volúmenes de información con tipos de datos diversos y con posibilidad de etiquetado.

Los métodos de recuperación y almacenamiento de información que proporciona la plataforma Pandas son de gran utilidad y brindan rapidez, confiabilidad y seguridad en el manejo de información.

Ahora reflexiona lo siguiente:

- ¿Puedes identificar apropiadamente las estructuras de datos de Pandas para manipular información tabular con diversos tipos de datos?
- ¿Te sientes capaz de utilizar métodos para la recuperación y almacenamiento de información tabular desde/hacia archivos con diversos formatos?

| Material adicional

Pandas

Pandas via. (septiembre de 2024). *Pandas documentation* [Documentación de Pandas]. Pandas. Recuperado el 14 de abril de 2025 de: <https://pandas.pydata.org/docs/>

Python

Python Software Foundation. (2025). *Python*. Recuperado el 14 de abril de marzo de 2025 de: <https://www.python.org/doc/>

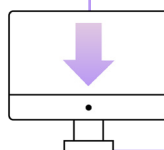
Orígenes de datos

Pandas vía. (2024). *IO tools (text, CSV, HDF5, ...)* [Herramientas de E/S (texto, CSV, HDF5, ...)]. Pandas. Recuperado el 14 de abril de 2025 de: https://pandas.pydata.org/docs/user_guide/io.html#io

Nota: Los enlaces a las páginas web incluidos en este apartado fueron consultados y verificados por última vez el 14 de abril del 2025 y su disponibilidad depende del autor.

Glosario

- **Archivo:** Conjunto de datos organizado y almacenado en un dispositivo digital.
- **Dataframe:** Estructura de datos de dos dimensiones de la plataforma Pandas que se concibe como un conjunto de Series para almacenar múltiples datos en forma tabular.
- **Datos tabulares:** Datos organizados en filas y columnas para poder estructurarlos y analizarlos cuando existen gran cantidad de ellos.
- **Formato de archivo:** Manera en la que se organiza y codifica la información en un archivo.
- **Serie:** Estructura de datos de una dimensión de la plataforma Pandas que permite almacenar una colección de datos indexados y/o etiquetados.



Descarga este documento en la sección "**Archivos adjuntos**" de la plataforma.